Assignment 2

Students IDs: 230041**225**, 230041**237**, 230041**251**

Introduction

This project simulates the operation of a simple Finite State Machine (FSM) based on cache controller. Caches are used to decrease the performance gap between a fast CPU and a slow main memory. The operation of a cache requires the use of an FSM (Finite State Machine) controller to facilitate the communication between the CPU, the cache, and the main memory. This report describes the design and implementation of a C++ simulator that simulates a direct-mapped, write-back, write-allocate cache using a 4 state FSM. The simulator was implemented in C++ and is made up of three main structural units:

Simple CPU (Request Queue): The CPU issues requests to the cache controller (read or write), and waits for the cache controller to signal to it that it is available to send another request (cpu_ready = true) before sending another request.

Memory Module: A very basic memory class that provides a simulation of the delay that the CPU will experience as it attempts to retrieve or write a block. The memory does not experience misses, but takes a specified number of clock cycles (latency = 2 cycles) to process read and writes, and maintains a busy flag and a counter to maintain its state.

Cache Controller: The cache is implemented as a 4-block direct-mapped cache (map<int, CacheLine> cache). Each CacheLine is storing a valid bits, dirty bits, and a tag. The controller uses integer memory addresses, calculating the index (with address % 4) and the tag (with address / 4) to map requests to the correct cache line. Interface signals (cpu_ready, mem_read, mem_write) are toggled based on the active FSM state.

Finite State Machine Implementation

The main component of the simulator is a four-state FSM, which is updated by the tick() function operation, every time the clock ticks. Transitions are taken directly from the book's state diagram.

IDLE State: The cache is waiting for the CPU to send a valid read or write request. Upon the arrival of a request, the interface signal cpu_ready is set to false, and the FSM assumes the COMPARE_TAG state to commence operations.

COMPARE_TAG State: The controller determines whether the requested block is in the cache (Valid is true and the Tag matches).

Hit: The first block is located; thus, the demand is met. In case the operation is a WRITE instruction, the dirty bit is set. Finally, the cpu_ready flag is set to true, and the FSM is returned to the IDLE state.

Miss (Clean): If the block is not found and the line is unmodified (dirty = false), the FSM enters the ALLOCATE state to get the new block.

Miss (Dirty): When the block is not found, but the current lines are modified (dirty = true), the FSM has to save the data first. It changes to WRITE_BACK.

WRITE_BACK State: The controller asserts the mem_write signal. It keeps on issuing requests to the memory module until the memory latency is fully met. Once complete, the line is marked as clean (dirty = false), and the fsm changes to ALLOCATE, whereby it fetches the new data.

ALLOCATE State: The controller asserts the mem_read signal. It waits for the memory to finish a simulation of read latency. Once the data arrives, the block is updated with the new tag, the valid bit is set to true, and dirty is false. The FSM then successfullly resolves the initial request by transitioning again to COMPARE_TAG.

Input/Output Analysis and FSM Reaction

To cover all FSM functionalities (Hit, Miss Clean, Miss Dirty), the following specific sequence of requests is given to the simulator: READ 0, WRITE 0, READ 4. Since the cache comprises 4 blocks, Address 0 corresponds to Index 0 (Tag 0) The Address 4 also falls under the Index 0 but has a Tag 1.

Action 1: READ 0 (Compulsory Miss) Cycle 1: The state is IDLE. READ 0 is issued by the CPU. The FSM changes to COMPARE_TAG while asserting cpu_ready = false.

Cycle 2: State is COMPARE_TAG. The cache checks the Index 0 and misses (Valid=F). The block is clean, and hence it is sent for ALLOCATE.

State is ALLOCATE in cycles 3-5. The controller assigns mem_read = true and waits for memory latency of 2 cycles. Finally, the controller updates the block (Valid=True, Tag=0) and changes the state back to COMPARE_TAG.

Cycle 6: State is COMPARE_TAG. The block is now a fact. A cache HIT is registered; The value of cpu_ready is set to true, and the FSM is directed to the IDLE state.

Action 2: WRITE 0 (Cache Hit) Cycle 7: The state is idle. It issues WRITE 0 (The FSM changes its states to COMPARE_TAG.

Cycle 8: State is COMPARE_TAG. The tag 0 is similar to the tag in storage. Cache HIT! Since it’s a write, the dirty bit in this block is set to True. The FSM shifts into IDLE state again.

Action 3: READ 4 (Dirty Miss and Write-back) Cycle 9: The State is IDLE. CPU issues READ 4; The FSM is thus changed to COMPARE_TAG.

Cycle 10: State is COMPARE_TAG. Tag 1 is yielded by Address 4. Hence, the stored tag is 0, which results in a MISS. Since during Action 2 the dirty bit was set true, the FSM must write back the data and changes to WRITE_BACK.

Cycles 11-13: WRITE_BACK is the state. The controller asserts mem_write = true. It waits for the 2 cycles of memory latency. After this, a transition to ALLOCATE is made, and the dirty bit is unasserted.

Cycles 14-16: State is ALLOCATE. To fetch the newly replaced block (Tag 1), the controller asserts mem_read = true. Finally, a new block (Tag 1) is fetched by asserting mem_read = true by the controller.

Cycle 17: State is COMPARE_TAG. Now the tag is proper. Lastly, “A cache HIT is registered, and the FSM successfully returns to IDLE”.

Code Snippet Output generated by Simulator:

```
=== STARTING CACHE FSM SIMULATION ===

--- Cycle 1 ---

Current State: IDLE

  -> CPU issued READ for Address 0 (Index:0, Tag:0)

--- Cycle 2 ---

Current State: COMPARE_TAG

  -> Cache MISS! (Valid: False, Dirty: False)

--- Cycle 3 ---

Current State: ALLOCATE

  -> Reading new block from Memory...

...

--- Cycle 10 ---

Current State: COMPARE_TAG
```

```
  -> Cache MISS! (Valid: True, Dirty: True)
--- Cycle 11 ---
Current State: WRITE_BACK
  -> Writing dirty block back to Memory...
```

GitHub Repository

The simulation code, complete with a descriptive README detailing how to build, run, and test the code sequence, can be accessed publicly at the following link:
https://github.com/StyreXxX/COA

Conclusion

The simulation proves well the execution logic of FSM cache controller. The program with the states of idling, tag comparison, allocation, and writing back correctly manages interface signals and latency interactions between the simulated CPU and Main Memory. Thus, using hits, clean misses, and dirty misses as triggers, the input sequence has been proved to result in dynamic transitions, indicating that the FSM adheres to the architectural specifications strictly.